RESEARCH PROPOSAL /
THUYẾT MINH ĐỀ CƯƠNG NGHIÊN CỨU

Nhóm 7

Research title	**Where Do Tiny Oriented Objects Lose Their Training Signal? Candidate-Region Geometry for Real-Time YOLO-OBB Detectors on AI-TOD-R**
Mentor	Hà Anh
Members *(up to 4 students)*	Võ Đức Nhật
	Trần Lợi Kiều Nhi
	Trần Nguyễn Minh Hải
	Nguyễn Tấn Tài

Abstract / Tóm tắt

Tiny objects in aerial images, such as vehicles, ships or people only a few pixels wide, are among the hardest targets for object detectors. In real-time one-stage detectors such as the YOLO family, the first failure happens during training: a ground-truth box smaller than the feature stride may contain no anchor point and then receives no training signal at all [1]. YOLO26 introduced Small-Target-Aware Label Assignment (STAL) to fix this, but STAL has only been evaluated on COCO with horizontal boxes, and its rule for oriented boxes changed three times in the public code during 2026 [1, 2]. This project studies how such candidate-region rules affect tiny oriented objects and whether a better rule helps. Research questions: (RQ1) how do YOLO11-OBB and YOLO26-OBB perform on AI-TOD-R, the oriented tiny-object benchmark with a mean object size of 10.6 px [3], when scored by its official evaluator and broken down by object size; (RQ2) how do the candidate rules change the number of candidates, where they fall and how often neighbouring objects compete for them; (RQ3) does a shape-preserving minimum-area rule (OA-STAL v2) with a conflict rule improve tiny-object accuracy at zero inference cost; (RQ4) do the findings transfer to a second dataset. The methodology combines a geometric analysis that predicts candidate counts, offline statistics computed from the labels, controlled ablations in one pinned code base, three random seeds, paired bootstrap confidence intervals and predictions registered before training.

Key words: tiny object detection; oriented object detection; label assignment; YOLO; aerial imagery; AI-TOD-R.

1. Introduction / Giới thiệu

1.1. Literature review / Tình hình nghiên cứu trong và ngoài nước

Why the problem matters in practice. Detecting tiny objects in aerial images supports vehicle detection, traffic-condition monitoring and maritime rescue [5]. More broadly, tiny-object detection is used for traffic monitoring and border surveillance [3] and for drone-scene analysis [17]. Many of these systems must run on drones or edge devices: real-time detection is described as powering applications "often on edge devices with tight latency and power budgets", and current

YOLO models export to TensorRT, ONNX and other embedded runtimes [1]. This is why we study *real-time* detectors rather than the slower two-stage pipelines that dominate tiny-object benchmarks.

Datasets. DOTA is the standard large-scale benchmark for oriented (rotated-box) detection in aerial images [19]. AI-TOD and AI-TOD-v2 focus on tiny objects but use horizontal boxes [16, 5]; AI-TOD-v2 defines the size bins used in this field: very tiny 2–8 px, tiny 8–16 px, small 16–32 px and medium 32–64 px [5]. SODA-A provides 2,513 large images with 872,069 oriented small objects [17]. **AI-TOD-R** adds oriented boxes to the AI-TOD images: 28,036 images, 752,460 oriented boxes in 8 classes, mean size 10.6 ± 4.9 px, "most objects … within the tiny scale (<16×16 pixels)", and often more than 100 objects per image [3]. It is the target benchmark of this project.

Label assignment for tiny objects. During training a detector must decide which feature-map positions (anchor points) learn which object; this step is called *label assignment*. Because Intersection-over-Union (IoU) changes sharply when a tiny box shifts by one or two pixels, several works replace it: NWD-RKA compares boxes as 2-D Gaussian distributions [5]; RFLA uses Gaussian receptive-field priors [6]; GCD proposes a scale-invariant Gaussian distance [7]. Their gains are large for two-stage detectors (RFLA lifts Faster R-CNN from 11.1 to 21.1 AP on AI-TOD) but small for one-stage detectors (RetinaNet from 8.7 to 9.1 AP with RFLA; +1.6 AP with NWD-RKA) [5, 6]. *Comment:* all three use horizontal boxes, and the small one-stage gains warn that assignment changes in YOLO-like detectors will produce small effects that need careful statistics.

Oriented tiny objects. On AI-TOD-R the best published method, DCFL, reaches 17.5 AP and only 6.5 AP on very tiny objects (AP_{vt}); a standard Oriented R-CNN reaches 11.2 AP but 0.6 AP on very tiny objects [3]. The authors trace the problem to assignment: "tiny-scale objects receive significantly fewer positive samples than larger-scale objects" because "the stride of prior locations in a typical single-stage detector is at least 8 pixels" [3]. Shape-aware assigners exist: SASM and CFA appear in the AI-TOD-R benchmark (11.4 and 12.4 AP) [3]; EARL samples inside an ellipse that follows the object's angle and aspect ratio [8]; SARFA-Net does the same and enlarges the sampling range of small objects by 1.1× [9]; RDCNet, built on YOLOv8, selects candidates in the object's rotated frame [10]. *Comment:* none of the last three was evaluated on a tiny-object dataset.

Real-time YOLO detectors and STAL. Modern YOLO models use task-aligned assignment (TAL) from TOOD [14]. YOLO26 [1] observed that "a tiny box may contain no valid anchor centers at all" and introduced **STAL**: while selecting candidates only, any box side shorter than 8 px is enlarged to 16 px; the true box is still used for regression. On COCO with YOLO11s, STAL raised AP from 46.6 to 46.8 and small-object AP from 29.0 to 29.6 (single runs) [1]. YOLO26s-OBB reaches 54.8 mAP50-95 on DOTA-v1.0 test [1]. The only YOLO-OBB numbers we found next to AI-TOD-R come from DIA-YOLO [4], which used 18,730 images instead of 28,036, a 4-channel (RGB + depth) input, up to 815 epochs and no size bins.

Table 1 summarises the works closest to this project, what they achieved and what they leave open.

Table 1. Key related works, their results and our comments (numbers as reported by the authors).

Work	What it does	Box type / data	Reported result	Comment for this project
AI-TOD-R + DCFL [3]	Oriented tiny benchmark; dynamic coarse-to-fine assignment with rotated Gaussian candidates	OBB; AI-TOD-R, SODA-A	17.5 AP, 6.5 AP_{vt} (best on AI-TOD-R)	Strong but not real-time; benchmark has no YOLO and no speed
NWD-RKA [5]	Gaussian Wasserstein similarity + ranking assignment; AI-TOD-v2	HBB; AI-TOD-v2	Faster R-CNN 12.8 → 21.4 AP; RetinaNet +1.6 AP	Large two-stage, small one-stage gains
RFLA [6]	Gaussian receptive-field prior + hierarchical assignment	HBB; AI-TOD	Faster R-CNN 11.1 → 21.1 AP; RetinaNet 8.7 → 9.1	Same pattern; no rotation
GCD [7]	Scale-invariant Gaussian distance for assignment and loss	HBB; AI-TOD-v2	Faster R-CNN 20.1 AP	Authors: verified mainly for horizontal detection
EARL [8]	Elliptical candidate region following angle and shape; nearest-anchor fallback	OBB; DOTA-like	—	No tiny-object dataset
SARFA-Net [9]	Shape-aware elliptical sampling; ×1.1 range for small objects	OBB; DIOR-R, DOTA, HRSC…	—	No tiny-object dataset
RDCNet [10]	YOLOv8-based; candidates in the rotated frame + fallback	OBB; DOTA, HRSC	—	Same family as ours, no tiny data
YOLO26 / STAL [1]	Enlarges tiny boxes during candidate selection	HBB experiments; COCO	AP 46.6 → 46.8; AP_S 29.0 → 29.6 (single runs)	Never tested on OBB or tiny data
DIA-YOLO [4]	YOLOv11-OBB-based model with depth input	OBB; "AITOD" 18,730 images	Compared with AI-TOD-R AP50 values	Different images, input, schedule; no size bins

Research in Vietnam. Within the 80 papers selected by our systematic literature review (3,039 records screened from OpenAlex, arXiv and conference proceedings) we did not identify a study by a Vietnamese team on oriented tiny-object detection. We did not search by author affiliation, so this is not evidence that no such work exists; the team will ask the mentor about related domestic work before the final version.

1.2. The limitation of current works / Những hạn chế của các nghiên cứu hiện tại

1) **STAL is untested where it matters most.** It was ablated only on COCO with horizontal boxes [1]. No published work evaluates it on oriented boxes or on any tiny-object dataset (checked on 08/10/2026).
2) **"STAL" is not one rule.** In the public Ultralytics code the rule for oriented boxes changed three times in 2026 (Table 2). Because the rule sits in the shared assigner, YOLOv8- and YOLO11-OBB trained with today's package use it too [2]. Results from different versions are therefore not comparable, and nobody has measured what each version does to tiny oriented objects.
3) **The benchmark has no real-time detector.** The AI-TOD-R table lists 27 configurations of slow detectors and reports no speed [3]. The one YOLO comparison (DIA-YOLO) uses a different image set, input and schedule, and reports no size bins [4].
4) **Shape-aware assignment was never tested on tiny oriented objects inside YOLO.** EARL, SARFA-Net and RDCNet were evaluated on DOTA-like data [8–10].

5) **The diagnosis is thin.** AI-TOD-R shows the positive-sample shortage only as figures [3]; no work measures how often neighbouring tiny objects compete for the same anchor points in dense scenes, although AI-TOD-R images often contain more than 100 objects [3].
6) **Evaluation settings differ silently.** The official AI-TOD-R evaluator keeps 1,500 detections per image, while Ultralytics keeps 300 by default [2, 15]; in dense images this cap alone can change recall.

Table 2. History of the oriented-box candidate rule in the Ultralytics code (release tags checked on 08/10/2026) [2].

Ultralytics release	Rule for oriented ground-truth boxes
≤ 8.4.7 (includes YOLO26 release 8.4.0, January 2026)	No size floor
8.4.8 – 8.4.38	Sides < 8 px set to 16 px, but applied to the real labels (a bug fixed in 8.4.39)
8.4.39 – 8.4.127	Sides < 8 px set to 16 px on a copy (= the published rule, "STAL-8")
≥ 8.4.128 (August 2026; current 8.4.174)	Sides < 16 px set to 16 px ("STAL-16")

1.3. The necessity of the research / Sự cần thiết tiến hành nghiên cứu

Problem. For tiny oriented objects, which candidate-region rule gives real-time detectors the most useful training signal, and why? **Central hypothesis:** for objects at the scale of AI-TOD-R, the *threshold* of the rule (8 vs 16 px) controls how many candidates an object receives, while its *shape* controls only where candidates fall and how often neighbouring objects compete for them; therefore a shape-preserving rule can help only elongated objects in specific neighbour layouts, and this can be predicted from the labels before training.

- **Originality.** We do not claim a new mechanism family: shape-aware regions already exist [8–10]. What is new is (i) the regime (oriented tiny objects, mean 10.6 px), (ii) the place (YOLO's candidate step, where every STAL version acts), and (iii) the evidence: a geometric account plus predictions fixed before the experiments.
- **Timeliness.** YOLO26 was released in January 2026 and its oriented-box rule last changed in August 2026 [2]; AI-TOD-R was published in IEEE TPAMI in 2025 [3].
- **Scientific significance.** The study explains a widely used training trick instead of only reporting a score, and it gives other researchers a fair, size-binned YOLO baseline on the hardest oriented tiny-object benchmark.
- **Practical significance.** The proposed change is training-only: model size, speed and deployment pipeline stay identical, so any gain transfers directly to edge deployment.

2. Research objectives / Mục tiêu của đề tài

1) **O1 (RQ1).** Build a reproducible benchmark of YOLO11-OBB and YOLO26-OBB on AI-TOD-R, scored by the official evaluator, with AP per size bin and per class and measured speed, anchored by an in-house reproduction of one published baseline.
2) **O2 (RQ2).** Measure, for each candidate rule, the zero-candidate rate, candidates and positives per object, conflicts between neighbouring objects and candidate placement, by size, aspect ratio and local density; relate these to per-object recall.
3) **O3 (RQ3).** Design and test OA-STAL v2 and two conflict rules against STAL-16, with predictions registered before training; report effects with 3 seeds and 95% confidence intervals.

4) **O4 (RQ4).** Check whether the findings transfer to a second dataset in proportion to the share of objects affected by the rules.
5) **O5.** Release code, configurations, prediction files and a written protocol so that the results can be reproduced.

3. Research scope / Phạm vi nghiên cứu

In scope	Out of scope
Oriented (rotated-box) detection of tiny objects in optical aerial images	Horizontal-box detection, segmentation, video tracking, SAR/infrared images
AI-TOD-R (main); SODA-A or DOTA-v2.0 train→val (second dataset)	Collecting or labelling new images
YOLO11-OBB and YOLO26-OBB (sizes n and s), one pinned Ultralytics code base; one reproduced MMRotate baseline	New backbones, attention modules or architectures; large models (l, x)
Training-time label-assignment rules: candidate region and conflict resolution	Changes to inference, loss functions or data augmentation research
Metrics: AP, AP50, AP75, AP per size bin and class (official AITODReval); latency, parameters, FLOPs	Claims of state-of-the-art accuracy or general claims about all YOLO models

4. Feasibility of research / Tính khả thi của đề tài

- **Data.** AI-TOD-R labels are public (`labels.zip`); images are rebuilt with the AI-TOD script after registering for the xView dataset [15, 16]. This is the first task of week 1 (Table 5).
- **Tools.** Everything is open source: Ultralytics (YOLO11/26-OBB) [2], the official AI-TOD-R code and evaluator `AITODReval` [15], MMRotate for the baseline. The team works in Python/PyTorch, which the DPL302m course already uses.
- **Compute.** Kaggle Notebooks give a free P100 or 2×T4 GPU, about 30 GPU-hours per week and 12-hour sessions [18]; training saves a checkpoint every epoch and resumes across sessions. Modal (≤ US$100) is the overflow. The time per run is estimated in Section 6 and replaced by a measured value in week 2.
- **Baselines.** The published AI-TOD-R table gives reference numbers [3]; we reproduce one row (RetinaNet-O, 7.3 AP) to check our evaluation pipeline (target: within ±1.0 AP).
- **Risks and fallbacks.** If xView access fails within two weeks, we use DOTA-v2.0 train→val (as in DCFL's own ablations [3]) and narrow the claims. If compute is short, we cut arms in a fixed order (Section 6). If OA-STAL v2 does not help, the benchmark and the diagnosis remain useful results, provided the diagnosis shows a clear mechanism (Section 7).

Table 3. Main risks, our judgement of their likelihood, and the planned response.

Risk	Likelihood (judgement)	Planned response
xView access or image rebuilding is delayed	unknown	Use DOTA-v2.0 train→val as a pre-declared fallback and narrow the claims
OA-STAL v2 gives no gain over STAL-16	medium–high: Proposition 1 shows shape changes only placement; one-stage assignment gains in the literature are ≤ 1.6 AP [5, 6]	Report the benchmark and the diagnosis; a null result is reported as a result, not hidden
Differences are within seed noise	medium	Three seeds, bootstrap confidence intervals, minimal effect fixed from measured seed variance
Our evaluation pipeline differs from the official one	medium	Reproduce a published baseline row before any comparison

Risk	Likelihood (judgement)	Planned response
The Ultralytics rule changes again	certain (already happened, Table 2)	One frozen code copy; all rules as explicit parameters; commit recorded
Closer prior work appears (e.g. [13])	unknown	Literature re-check in week 13; no "first" claims
GPU quota is not enough	medium	Fixed cutting order (Section 6); Modal overflow up to US$100

Status: no experiment has been run yet. The plan becomes fully runnable after the week-2 checkpoint (data rebuilt, evaluator validated, speed measured).

5. Approach and Method / Cách tiếp cận và phương pháp nghiên cứu

5.1. Overall framework

Figure 1 shows the system. Data are prepared once; every experiment trains the same YOLO-OBB model in one frozen copy of the Ultralytics code and changes **only** the label assigner (red block). Predictions are always scored by the official evaluator. The diagnosis (green block) runs first on the labels alone and fixes the predictions that the training experiments must confirm or reject.

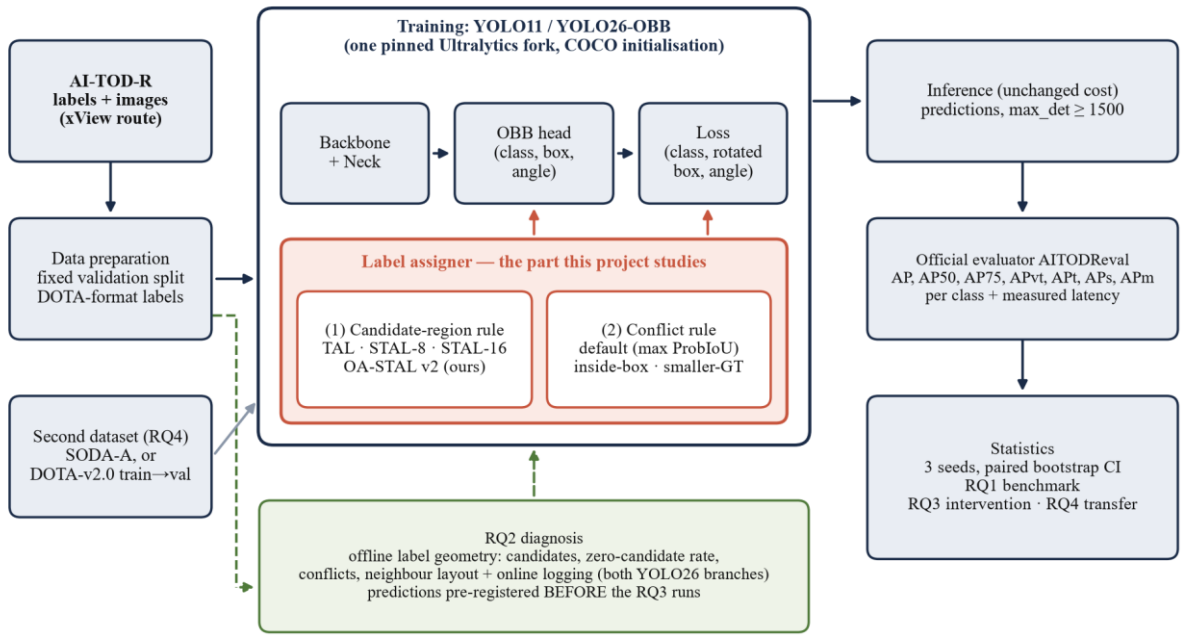


Figure 1. Research framework. Only the label assigner changes between experiments; inference is identical.

5.2. How YOLO chooses training samples

For each ground-truth (GT) box, YOLO's task-aligned assigner [14, 2] works in three steps: (1) **candidate selection** — keep the anchor points whose centres fall inside a candidate region around the GT; (2) **ranking** — score each candidate by classification confidence and predicted-box overlap and keep the best ones as *positives*; (3) **conflict resolution** — if one anchor point is a positive for several GTs, give it to the GT with the highest overlap. Anchor points lie on grids with spacing (stride) 8, 16 and 32 px. A 4×12 px object can therefore fall between grid points and receive no candidate at all. STAL enlarges the candidate region in step 1 only. This project studies step 1 and step 3.

5.3. Candidate-region rules and a geometric analysis

Figure 2 compares the four rules on the same rotated 4×12 px object. TAL uses the object itself; STAL-8 enlarges only the 4-px side, which turns the region by 90° relative to the object; STAL-16 turns it into a 16×16 square; our OA-STAL v2 enlarges the object *similarly* (same centre, angle and shape) until it has the same area as STAL-16.

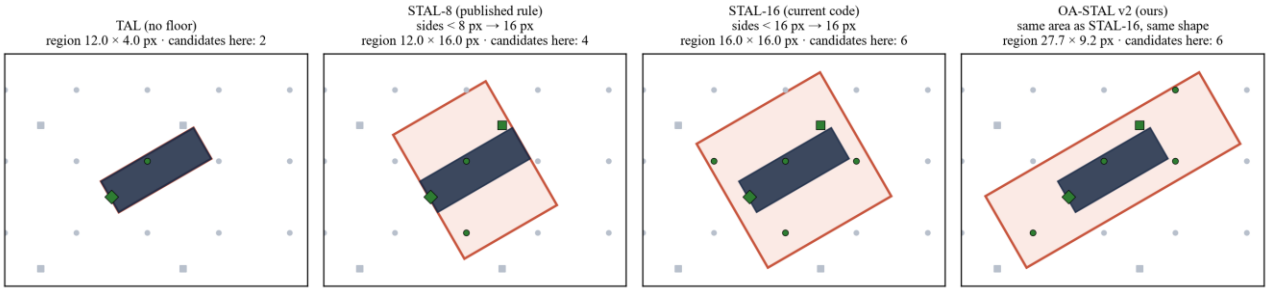


Figure 2. Candidate regions of the four rules for one rotated 4×12 px object (computed for one placement on the anchor grid).

Proposition 1 (expected number of candidates). If a candidate region R is placed at a random position on the anchor grids, the expected number of anchor centres inside it is

$$E[N] = \text{area}(R) \times (1/8^2 + 1/16^2 + 1/32^2) = (21/1024) \times \text{area}(R) \approx 0.0205 \times \text{area}(R).$$

Why: on a grid with spacing s, a randomly placed region of area A contains on average A/s^2 grid points; rotation does not change area, and the three grids are added because the assigner pools them. *Check:* an independent simulation of the code's rotated test gives 5.2–5.3 candidates for a 16×16 region (predicted 5.25) and 2.05 for a 10×10 box (predicted 2.05). Three consequences guide the whole study:

- **Tiny boxes can be missed.** Since N is a whole number, $P(N = 0) \geq 1 - 0.0205 \times \text{area}$: a 4×4 px box has at least a 67% chance of receiving no candidate under plain TAL (simulation: 67.1%). This is the failure STAL fixes.
- **Same area means same count.** Two rules with equal region area give the same expected number of candidates. So a shape-preserving rule cannot add candidates compared with STAL-16; it can only change *where* they fall and *how often neighbours compete*. Our hypothesis is therefore about placement and conflicts, not coverage.
- **The threshold matters more than the shape.** For a 10×10 px object, STAL-8 changes nothing (2.05 expected candidates) while STAL-16 gives 5.25, about 2.5 times more. In AI-TOD-v2, 73.4% of objects are in the 8–16 px bin [5], so the 8-vs-16 choice is a first-order factor that must be controlled.

The same simulation shows the trade-off that the experiments will test: for two parallel 4×12 px objects lying side by side, the share of candidates claimed by both falls from 0.63 (STAL-16) to 0.34 (similar enlargement); for objects lying end to end it rises from 0.12 to 0.49. Which layout dominates in AI-TOD-R can be counted from the labels before training, so the expected sign of the effect is fixed in advance.

5.4. Proposed method: OA-STAL v2 and conflict rules

Let a GT be (x, y, w, h, θ) with long side w ≥ h. OA-STAL v2 is applied only if √(wh) < 16 px; then

$$k = 16 / \sqrt{(wh)}, \quad h' = \max(k \cdot h, 8, h), \quad w' = \max(\min(k \cdot w, \max(w, 2 \cdot 16)), w).$$

The candidate region is (x, y, w', h', θ); scoring, targets and regression still use the true box, exactly as in STAL [1]. In words: enlarge the box with the same shape to an area of 16×16; never make the short side thinner than 8 px (protects very thin objects); cap the long side at 32 px unless the object is already longer; never shrink anything. Its properties can be checked by substitution: the region always contains the true box, so it never gives fewer candidates than plain TAL; square boxes give exactly the STAL-16 region; the shape is kept exactly for aspect ratios up to 4; inference cost is unchanged. Two conflict rules are tested against the default (highest overlap): **inside-box** (an anchor inside exactly one true box goes to that box) and **smaller-GT** (the smaller object wins, as in RFLA [6]). A minimal implementation sketch (to be unit-tested before any training):

```
def candidate_region(gt, mode):            # gt = (x, y, w, h, theta), w >= h
    x, y, w, h, t = gt
    if mode == 'TAL':     return gt
    if mode == 'STAL-8':  return (x, y, 16 if w < 8 else w, 16 if h < 8 else h, t)
    if mode == 'STAL-16': return (x, y, max(w, 16), max(h, 16), t)
    if mode == 'OA-STAL':
        if (w * h) ** 0.5 >= 16: return gt                # not tiny: unchanged
        k = 16 / (w * h) ** 0.5
        h2 = max(k * h, 8, h)                             # short-side floor
        w2 = max(min(k * w, max(w, 32)), w)               # conditional long-side cap
        return (x, y, w2, h2, t)
```

5.5. Experimental design

Evaluation. Predictions are exported to DOTA text format and scored only by the official `AITODReval` (IoU 0.50:0.05:0.95, 1,500 detections per image, size bins as in [5]); Ultralytics' detection cap is raised to ≥ 1,500. Hyper-parameters are tuned on a fixed validation split cut from the training set (10% of images), because the official configuration uses the test set for validation [15]; the test set is used once per final model. All YOLO models start from COCO weights, because AI-TOD is partly built from DOTA images [5] and the released YOLO26-OBB weights were trained on DOTA [2].

RQ1. YOLO11s-OBB, YOLO26n-OBB and YOLO26s-OBB at 800×800 input, with two clearly separated recipes: a short flip-only recipe close to the published benchmark and the standard Ultralytics recipe; three seeds for YOLO26n. **RQ2.** Offline, from labels only: for every object and every rule, the number of candidates, zero-candidate cases, candidates shared with neighbours, the neighbour layout and the share of candidates outside the object. Online, during training: positives per object for both YOLO26 output branches, and per-object recall on the validation set. **RQ3.** The arms in Table 4, each with three seeds on YOLO26n. **RQ4.** Arms A0, A2 and A4 on the second dataset, reported against the share of objects whose rule differs.

Table 4. Experimental arms (all in one pinned code base; only the assigner changes).

Arm	Candidate rule	Conflict rule	Purpose
A0	TAL (no floor)	default	Lower bound; shows the zero-candidate failure
A1	STAL-8 (published rule)	default	Published version
A2	STAL-16 (current code)	default	Main comparator

Arm	Candidate rule	Conflict rule	Purpose
A3	TAL + nearest-anchor fallback	default	Cheap control used by EARL / RDCNet [8, 10]
A4	OA-STAL v2	default	Main proposal
A5	Similar enlargement, side trigger	default	Separates threshold from shape (vs A2)
A6	OA-STAL v2	inside-box	Effect of the conflict rule
A7	STAL-16	inside-box	Conflict rule without shape change

Statistics and pre-registration. The main comparison (A4 vs A2 on AP_t and AP_{vt}) is fixed in advance. Each arm is run with three seeds; we report mean ± standard deviation and a paired, image-level bootstrap 95% confidence interval (10,000 resamples). A change is called an improvement only if the interval excludes zero **and** the gain appears where the label statistics predicted (elongated classes such as ship and bridge, and the dominant neighbour layout). Before any RQ3 run, a dated pre-registration file records these predictions and the analysis code; any later deviation is logged with its reason.

5.6. Threats to validity and how the design handles them

- **Internal validity (is the change really caused by the rule?).** All arms share one frozen code copy, one data split, one schedule and the same seeds; only the assigner differs. The threshold and the shape are separated by arm A5, so a gain cannot be credited to shape when it comes from the threshold.
- **Construct validity (do we measure the right thing?).** AP per size bin from the official evaluator is the outcome; candidate counts alone are not treated as success, because more positives can also mean more conflicts or more false positives. Per-object recall links the diagnosis to the outcome.
- **Statistical validity.** Three seeds, paired bootstrap intervals and one pre-declared main comparison limit false discoveries; secondary comparisons are reported without significance claims.
- **External validity (does it generalise?).** AI-TOD-R is dominated by vehicles (662,929 of 752,460 objects [3]); results are therefore also reported per class, and RQ4 checks a second dataset. We will not claim that findings hold for other detector families or for horizontal boxes.
- **Reproducibility.** Code version, configurations, seeds, the pre-registration and prediction files are released, so that every table can be regenerated.

6. Research plan / Kế hoạch thực hiện nghiên cứu

The plan covers 14 weeks (12/10/2026 – 17/01/2027). Members are listed as M1–M4 and will be assigned by the team.

Table 5. Research plan (M1–M4 = team members).

No.	Date	Task	Output	Person in charge
1	12/10/2026 – 18/10/2026	Register xView, rebuild AI-TOD-R images, pin the Ultralytics fork, write unit tests for the four rules	Dataset ready; tests passing	M1, M2
2	19/10/2026 – 25/10/2026	Pilot runs on Kaggle (P100, 2×T4); prediction converter + AITODReval; reproduce RetinaNet-O baseline	Measured speed; evaluator within ±1.0 AP; go/no-go	M2, M3

No.	Date	Task	Output	Person in charge
3	26/10/2026 – 01/11/2026	Offline label analysis (RQ2): candidates, conflicts, neighbour layout; write pre-registration	Statistics report; dated pre-registration	M1, M4
4	02/11/2026 – 22/11/2026	RQ1 benchmark: YOLO11s/26n/26s-OBB, two recipes, seeds	Benchmark table per size bin and class	M2, M3
5	16/11/2026 – 13/12/2026	RQ3 arms A0–A7 × 3 seeds with online diagnostics	Ablation table with confidence intervals	All
6	14/12/2026 – 27/12/2026	RQ4 on the second dataset (SODA-A or DOTA-v2.0)	Transfer table	M3, M4
7	28/12/2026 – 03/01/2027	Analysis: link diagnosis to accuracy; error analysis by class and layout	Figures and findings	M1, M4
8	04/01/2027 – 10/01/2027	Re-check new literature; write the report / paper draft	Draft manuscript	All
9	11/01/2027 – 17/01/2027	Internal review with mentor; release code, configs and predictions	Final report + public repository	All

Computational Resource Requirements

- **GPU:** Kaggle free tier (P100 or 2×T4, ~30 GPU-hours/week, 12-hour sessions, 20 GB saved output, 4 CPU cores, 29 GB RAM) [18]; Modal pay-as-you-go up to US$100 as overflow.
- **Estimated cost per run** = images × epochs / (images per second × 3,600). With about 14,000 training images (estimate, to be replaced after download) and 100 epochs: 3.9 GPU-hours at 100 img/s, 7.8 at 50 img/s, 15.6 at 25 img/s. Ablations use the small YOLO26n model and shorter schedules.
- **Total plan:** about 9 runs (RQ1) + 24 (RQ3) + 9 (RQ4) + pilots. If the measured speed implies more than 10 weeks of quota, we cut in this order: RQ4 to two arms, A5, then A6/A7.
- **Storage and software:** about 30–50 GB for images, labels and checkpoints (private Kaggle dataset); Python 3, PyTorch, Ultralytics 8.4.x (pinned), MMRotate/AITODReval.

7. Expected results / Dự kiến kết quả đề tài

- **Theoretical:** a simple geometric account (Proposition 1) of what candidate-region rules can and cannot change for tiny objects, tested against real label statistics.
- **Methodological:** the first evaluation, to our knowledge within the sources we searched, of all STAL versions on oriented tiny objects; a fair, size-binned YOLO-OBB benchmark on AI-TOD-R with released prediction files; a training-only rule (OA-STAL v2 + conflict rule) tested against pre-registered predictions. Null results will be reported as results.
- **Practical:** because the change is training-only, any gain carries over to existing YOLO deployment pipelines (TensorRT/ONNX on edge devices [1]) without extra cost; the benchmark helps practitioners choose a real-time detector for traffic monitoring, maritime search and similar drone or satellite tasks [3, 5].

Table 6. Deliverables per objective and how success is judged.

Objective	Deliverable	Success criterion (decided in advance)
O1 Benchmark	Table of YOLO11/26-OBB on AI-TOD-R (AP per size bin and class, speed); prediction files	Evaluator reproduces the RetinaNet-O row within ±1.0 AP; 3 seeds for YOLO26n
O2 Diagnosis	Label-geometry report; training logs of positives per object; recall analysis	Measured candidate counts match Proposition 1; a clear link (or a documented absence of link) to recall
O3 Intervention	OA-STAL v2 and conflict rules in the pinned code; ablation table A0–A7	Main comparison A4 vs A2 judged by 95% bootstrap CI and the pre-registered pattern

Objective	Deliverable	Success criterion (decided in advance)
O4 Transfer	Results on the second dataset	Effect compared with the share of objects whose rule differs
O5 Reproducibility	Public repository: code, configs, seeds, pre-registration, predictions	Another team member can re-run one arm from the instructions

- **Publication target:** a Scopus-indexed conference or Q4 journal, conditional on a clean evaluator-matched benchmark, a clear diagnosis finding and confidence intervals on the main comparison; this is an assignment-level contribution, so we do not target top-tier venues.

References

[1] G. Jocher et al., "Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models," arXiv:2606.03748, 2026. https://arxiv.org/abs/2606.03748

[2] Ultralytics, ultralytics package, release 8.4.174 (06/10/2026), file ultralytics/utils/tal.py. https://github.com/ultralytics/ultralytics

[3] C. Xu et al., "Oriented Tiny Object Detection: A Dataset, Benchmark, and Dynamic Unbiased Learning," IEEE TPAMI, 2025. https://doi.org/10.1109/tpami.2025.3634161

[4] Kim and Park, "DIA-YOLO: Diffusion-Based Involution Attention YOLO for Overhead Imagery," IEEE Access, vol. 13, 2025. https://doi.org/10.1109/ACCESS.2025.3603214

[5] C. Xu et al., "Detecting tiny objects in aerial images: A normalized Wasserstein distance and a new benchmark," ISPRS J. Photogramm. Remote Sens., vol. 190, pp. 79–93, 2022. https://doi.org/10.1016/j.isprsjprs.2022.06.002

[6] C. Xu et al., "RFLA: Gaussian Receptive Field Based Label Assignment for Tiny Object Detection," ECCV, 2022. https://doi.org/10.1007/978-3-031-20077-9_31

[7] Guan et al., "Gaussian Combined Distance: A Generic Metric for Object Detection," IEEE GRSL, 2025. https://doi.org/10.1109/lgrs.2025.3531970

[8] "EARL: Elliptical Distribution aided Adaptive Rotation Label Assignment," IEEE TGRS, vol. 61, 2023, arXiv:2301.05856. https://arxiv.org/abs/2301.05856

[9] "SARFA-Net: Shape-Aware Label Assignment and Refined Feature Alignment for Arbitrary-Oriented Object Detection," IEEE JSTARS, vol. 18, 2025. https://doi.org/10.1109/JSTARS.2025.3532039

[10] "RDCNet: Rotation-Aware Feature Representation and Aspect-Adaptive Label Assignment," IEEE JSTARS, vol. 19, 2026. https://doi.org/10.1109/JSTARS.2026.3687230

[11] "MFN-Net: multi-scale adaptive fusion and flexible label assignment for small object detection in RS images," Remote Sensing Letters, 17, 2026 (abstract read only). https://doi.org/10.1080/2150704X.2026.2667240

[12] "Aspect-Ratio Aware Dynamic Label Assignment for Oriented Object Detection," IEEE GRSL, 21, 2024 (abstract read only). https://doi.org/10.1109/LGRS.2024.3432810

[13] "Cross-Domain Collaborative Enhancement for Tiny Object Detection in Remote Sensing Images," J. Electronics & Information Technology, 2026 (content not yet verified). https://doi.org/10.11999/JEIT260317

[14] C. Feng et al., "TOOD: Task-aligned One-stage Object Detection," ICCV, 2021. https://doi.org/10.1109/iccv48922.2021.00349

[15] AI-TOD-R project page and code (AITODReval evaluator), accessed 08/10/2026. https://github.com/Chasel-Tsui/AI-TOD-R

[16] J. Wang et al., "Tiny Object Detection in Aerial Images," ICPR, 2020 (published 2021). https://doi.org/10.1109/ICPR48806.2021.9413340

[17] G. Cheng et al., "Towards Large-Scale Small Object Detection: Survey and Benchmarks," IEEE TPAMI, 2023. https://doi.org/10.1109/tpami.2023.3290594

[18] Kaggle documentation: Notebooks and Efficient GPU Usage, accessed 08/10/2026. https://www.kaggle.com/docs/notebooks

[19] J. Ding et al., "Object Detection in Aerial Images: A Large-Scale Benchmark and Challenges," IEEE TPAMI, 2022. https://doi.org/10.1109/tpami.2021.3117983